ASSIGNMENT OF SOFTWARE-BASED SYSTEM DESIGN (SBSD)

SUBJECT: SOFTWARE-BASED SYSTEM DESIGN

SUBJECT CODE: EE 385

DEPARTMENT: ELECTRICAL ENGINEERING

SEMESTER: 6th

SUBMITTED BY:
NAME: Imon Shikha Patiri (2313048)
: Sneha (2313075)
: Evangeline Hasda (2313078)

Table of Contents

1. DC Motor……1
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

2. PWM Generator…….2
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

3. Speed Control of Squirrel-Case Induction Motor……3
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

4. Multilevel Inverter……4
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

5. Solar PV Panel Model……5
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

6. Open-loop and closed-loop position control of a DC Motor……6
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

7. Wind generator…….7
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion
8. Buck and Boost Converter……8
    - Title
    - Introduction
    - Procedure
    - Results
    - Discussion

1. DC Motor

TITLE: Modelling of DC Motor using MATLAB Simulink.

INTRODUCTION:

The DC motor is an essential component of sophisticated control systems as it is capable of converting electrical power into precise mechanical rotational motion. The DC motor in Simulink employs electrical and mechanical equations in the depiction of the motor's dynamic performance. This allows the user to simulate the motor's operational performance.
The DC motor's electrical side follows Kirchhoff's voltage law:
V = Raia+La(dia/dt)+eb,
where eb is back emf, eb=Kbw, Ra is the armature resistance, La is inductance, and w is angular speed.
Mechanically, the torque balance gives :
Tm=J(dw/dt)+Bw+TL,
With Tm=Ktia(torque constant), J as inertia, B as damping, and TL as load torque. The DC motor Simulink model usually displays a transient response in speed and current before they reach steady-state. It validates the parameters, such as speed at no load and stall torque. Changes in load cause speed reductions, and torque increases proportionally.

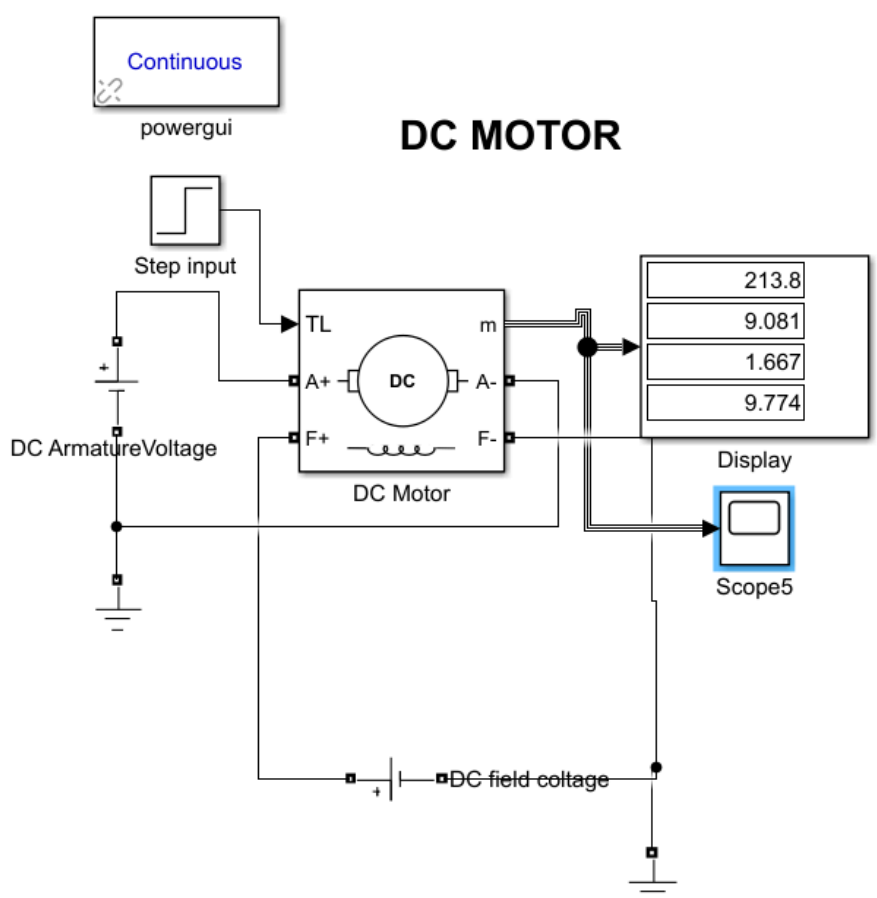
PROCEDURE:

Steps in the formation of the model:
The model mainly consists of Powergui, Step Input, DC Voltage Sources (armature & field), DC Motor block, Ground, Display, and Scope.

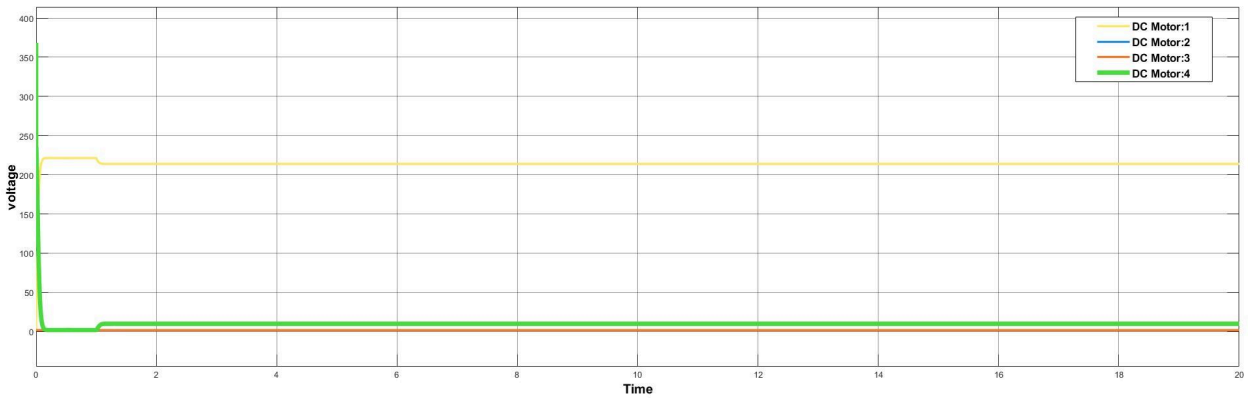
1. Open MATLAB. Type Simulink in the Command Window, and click Blank Model to create a new model.
2. Go to the Library Browser. Open: Simscape → Electrical → Specialized Power Systems → Machines
3. Select the DC Machine block.
4. We also add: DC Voltage Source, Display, Mechanical Load (Torque), Ground, Powergui, Scope, step input.

5. Double-click the DC Machine block and select and change the parameters: the motor type, Armature resistance, Field resistance, rated voltage, rated speed, Moment of inertia, Friction coefficient.
6. Connect the DC Voltage Source to the armature terminals and another DC voltage source to the field, and connect the mechanical load torque to the shaft. We also add Ground properly.
7. Connect Scope to: Armature current, Speed, and Electromagnetic torque.
8. Insert a Powergui block (mandatory) and change it to discrete.
9. Save the file and click Run.
10. Observe output in Scope: Speed vs Time, Current vs Time, Torque vs Time.

RESULTS:



CONCLUSION:



The Simulink model of the DC motor successfully demonstrates the relationship between applied voltage, armature current, speed, and torque. The results show that when the input voltage increases, the motor speed also increases until it reaches a steady state. The simulation verifies the theoretical behaviour of a DC motor and helps in understanding its dynamic response.

2. PWM Generator

TITLE: Modelling of PWM using MATLAB Simulink.

INTRODUCTION:

PWM in Simulink generates variable-width pulses by comparing a control signal (modulating wave) against a high-frequency carrier (typically triangular or sawtooth), controlling average power delivery to loads like DC motors without dissipative elements.

The duty cycle D(fraction of period pulses are high) determines output: average voltage equals $V_{avg}=D \cdot V_{peak}$

, where V_{peak} is the supply. For a modulating signal $m(t)$ (0 to 1) versus carrier $c(t)$, PWM output is high when $m(t)>c(t)$, yielding $\bar{y}=Dy_{max}+(1-D)y_{min}$.

Use PWM Generator (Simscape) or manual setup with Relational Operator: input carrier frequency (e.g., 10 kHz), period, and duty from PID controller or sine wave for SPWM. Outputs connect to H-Bridge or motor voltage port; higher carrier frequency reduces ripple but increases switching losses.

PROCEDURE:

Steps in the formation of the model:

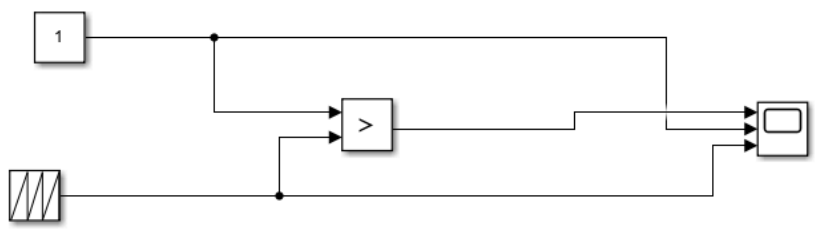
The model mainly consists of a Constant block, a Repeating Sequence (carrier) block, a Sine Wave block, a Relational Operator (>) block, and a Scope.

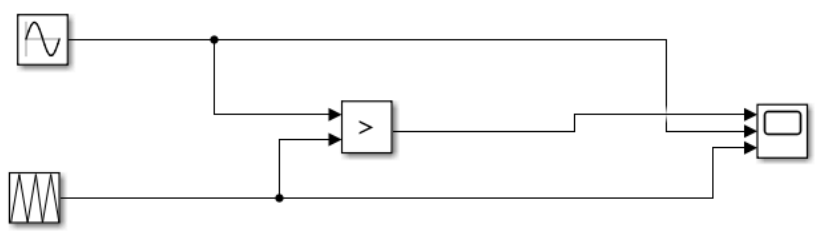
1. Open MATLAB. Type Simulink in the Command Window, and click Blank Model to create a new model.
2. In the Simulink Library Browser, open Simulink → Sources, Simulink → Math Operations, Simulink → Sinks, and optionally Simscape → Electrical → Specialized Power Systems → Power Electronics if you are using the PWM Generator block.
3. Add the following blocks: Sine Wave (reference/modulating signal), Repeating Sequence (carrier/triangular wave), Relational Operator (>), Scope, and Constant (if a DC reference is required).
4. Set the Sine Wave (reference signal) to Amplitude 1, Frequency 50 Hz, Sample time 0 (continuous); configure the Repeating Sequence (carrier wave) with Time values [0 0.0005 0.001] and Output values [0 1 0] to

create a triangular waveform; and set the Relational Operator block's operator to ">".

5. Connect the Sine Wave output to one input of the Relational Operator, the Repeating Sequence output to the other input, then connect the Relational Operator output to the Scope and also feed the Sine and Carrier signals to the Scope (using a Mux if needed) to observe their comparison.
6. Save the file.
7. Click Run, open the Scope, and observe the sine reference wave, the triangular carrier wave, and the resulting PWM output pulses.

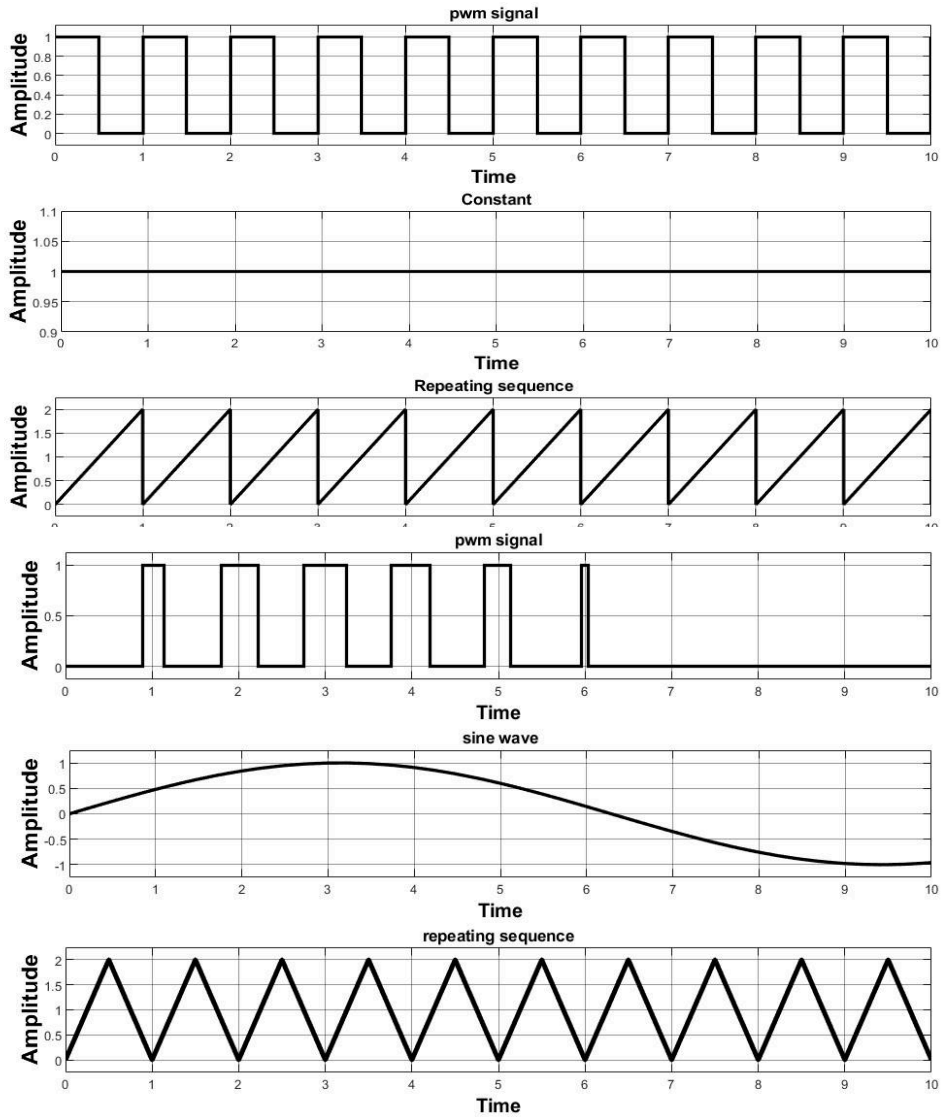
RESULTS:





PWM GENERATOR

CONCLUSION:



The PWM generator simulation successfully produces pulse-width-modulated signals with controllable duty cycles. The results show that by changing the duty cycle, the width of the pulses changes while the frequency remains constant. This experiment confirms that PWM is an effective technique for controlling power delivered to electrical devices.

3. Speed Control of Squirrel-Case Induction Motor

TITLE: Modelling of Speed Control of Squirrel Case Induction Motor using MATLAB Simulink.

INTRODUCTION:

Speed control of a squirrel cage induction motor in Simulink is based on modelling the motor's electromagnetic dynamics and then regulating its stator voltage and frequency (or stator currents) so that the rotor speed tracks a reference under different loads.

The synchronous speed of a three-phase induction motor is set by supply frequency f and pole pairs p, while the actual rotor speed is slightly lower due to the slip needed to produce torque. Scalar V/f control keeps the ratio of stator voltage to frequency approximately constant so that the air-gap flux remains nearly constant; changing frequency then changes speed with acceptable torque capability. Expected Simulation Behaviour: For step changes in speed reference, the motor speed should rise smoothly to the new setpoint, with electromagnetic torque showing a transient spike and then settling as slip stabilises. Compared with open-loop V/f, closed-loop scalar or vector control in Simulink yields better dynamic performance, smaller steady-state speed error, and improved efficiency across load variations for the squirrel cage induction machine.

PROCEDURE:

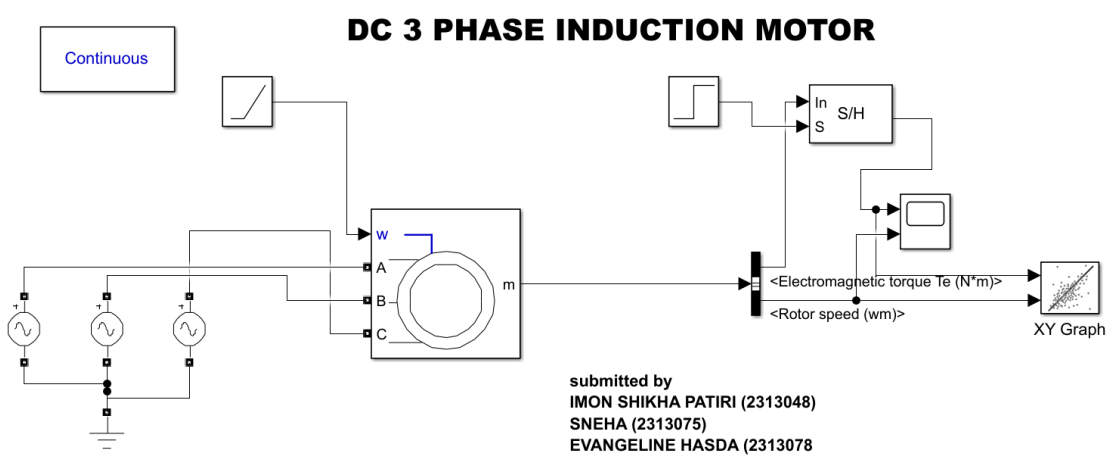
Steps in the formation of the model:

The model mainly consists of Powergui (Continuous), Step block, Gain block, Three-Phase Source, Asynchronous Machine (Induction Motor) block, Bus Selector, Display, and Scope.

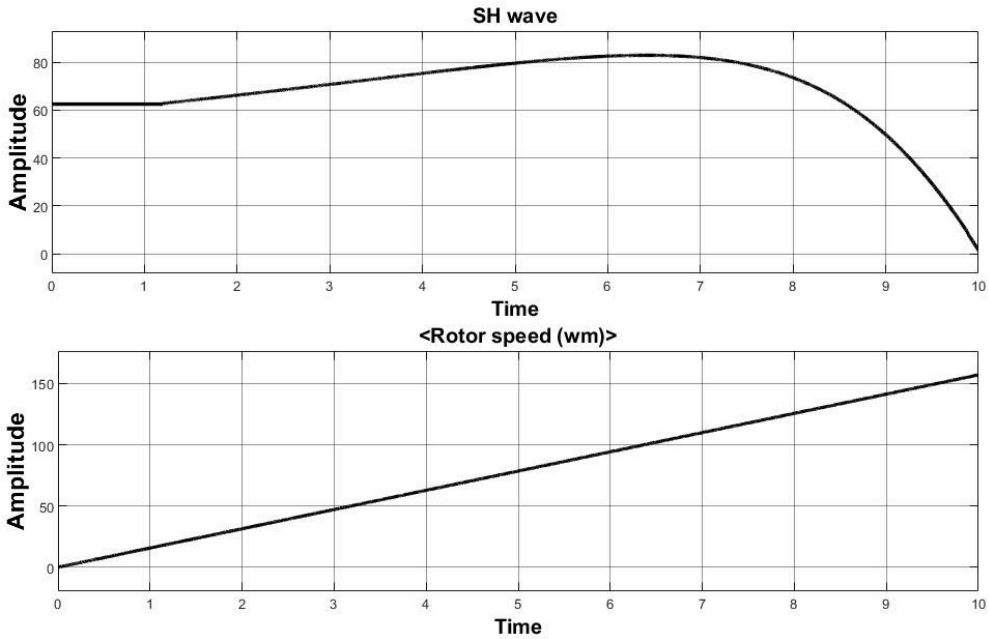
1. Open MATLAB. Type Simulink in the Command Window, and click Blank Model to create a new model.
2. In the Library Browser, open Simscape → Electrical → Specialized Power Systems → Machines, Simscape → Electrical → Specialized Power Systems → Power Electronics, Simulink → Sources, Simulink → Sinks, and Simscape → Utilities.

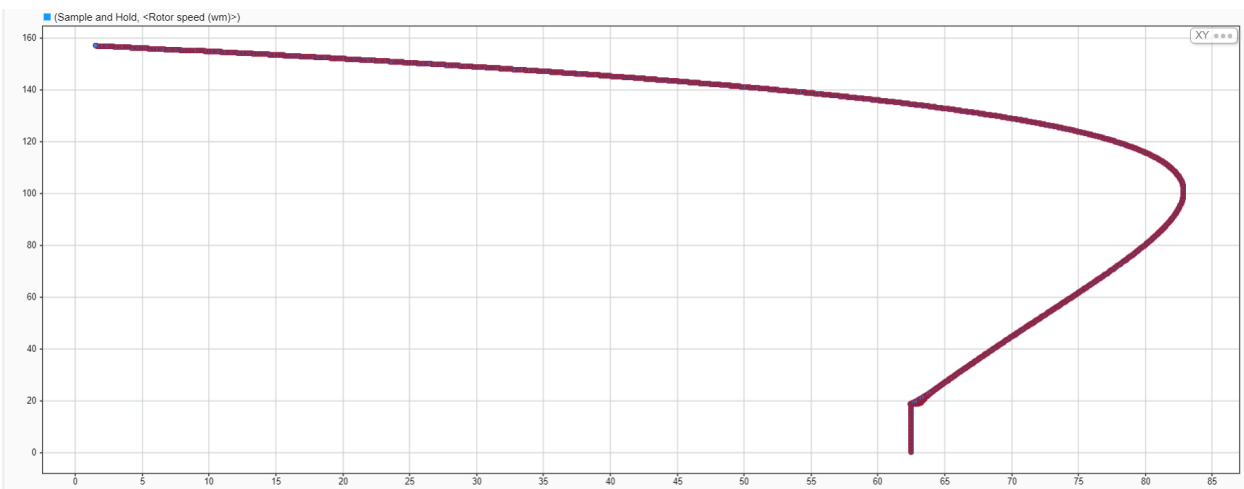
3. Add the following blocks: Asynchronous Machine (Squirrel Cage), Three-Phase Source, Universal Bridge (IGBT-based), PWM Generator (3-phase, 2-level), DC Voltage Source, Mechanical Load (Torque), Scope, Powergui, and Ground.
4. Configure the induction motor by double-clicking the Asynchronous Machine block, entering the motor parameters (rated power, rated voltage, frequency 50 Hz, stator and rotor resistances and inductances, inertia J, friction coefficient B), selecting rotor type as Squirrel Cage, and clicking Apply.
5. For basic speed control using V/f (Voltage/Frequency) control, use a variable frequency source via an inverter by connecting the DC source to the Universal Bridge, the Universal Bridge to the motor, and configuring the PWM Generator with a suitable carrier frequency (e.g., 2 kHz) and a connected reference frequency input.
6. Connect the 3-phase output of the inverter to the motor stator terminals, apply the mechanical load torque to the shaft, connect measurement outputs for speed, stator current, and electromagnetic torque, and include a mandatory Powergui block in the model.
7. Save the file.
8. Click Run, open the Scope, and observe speed vs time, torque vs time, and stator current.

RESULTS:



CONCLUSION:





XY graph

The simulation demonstrates that the speed of the squirrel-cage induction motor can be effectively controlled by varying input parameters such as supply frequency or voltage. The results show smooth acceleration and stable operation at steady state. This validates the effectiveness of Simulink for studying induction motor performance and control.

4. Multilevel Inverter

TITLE: Modelling of Multilevel Inverter using MATLAB Simulink.

INTRODUCTION:

A multilevel inverter, when modelled in Simulink, generates a staircase AC voltage from various DC voltage levels by using a series of power electronic devices and appropriate PWM logic, resulting in an output voltage that is close to a sinusoidal waveform with less harmonic distortion and less stress on the power electronic devices compared to a two-level inverter.

A multilevel inverter is a type of inverter that produces an output voltage that is close to a sinusoidal waveform by using various discrete voltage levels, such as 3-level, 5-level, 7-level, and so on. As a result, the THD and dv/dt ratings of the inverter output voltage decrease as the voltage level increases. A multilevel inverter model in Simulink can take various forms, such as a diode-clamped, flying capacitor, and cascaded H-bridge inverter, based on the method of obtaining various voltage levels in the inverter output.

A multilevel inverter, when modelled in Simulink, is constructed by using a series of bridge structures, i.e., H-bridges, that are supplied from separate DC sources. Each switch in the inverter is then driven by appropriate gating pulses that are generated by an SPWM or SVPWM logic. Comparator logic is used to generate appropriate gating pulses in the inverter.

PROCEDURE:

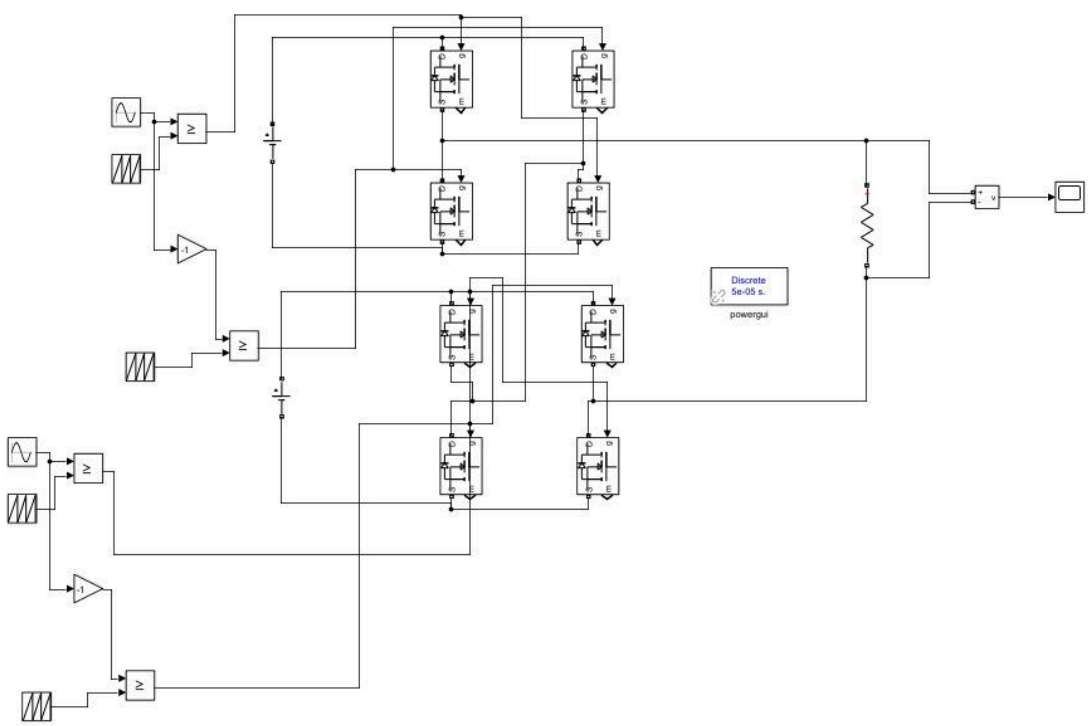
Steps in the formation of the model:

The model mainly consists of Powergui (Discrete), Sine Wave block, Repeating Sequence (carrier) block, Relational Operator (≥) block, Gain block, IGBT/Diode blocks, DC Voltage Source, Resistor (R Load), Voltage Measurement block, and Scope.

1. Open MATLAB. Type Simulink in the Command Window, and click Blank Model to create a new model.
2. In the Library Browser, open Simscape → Electrical → Specialized Power Systems → Power Electronics, Simulink → Sources, Simulink → Sinks, and Simscape → Utilities.

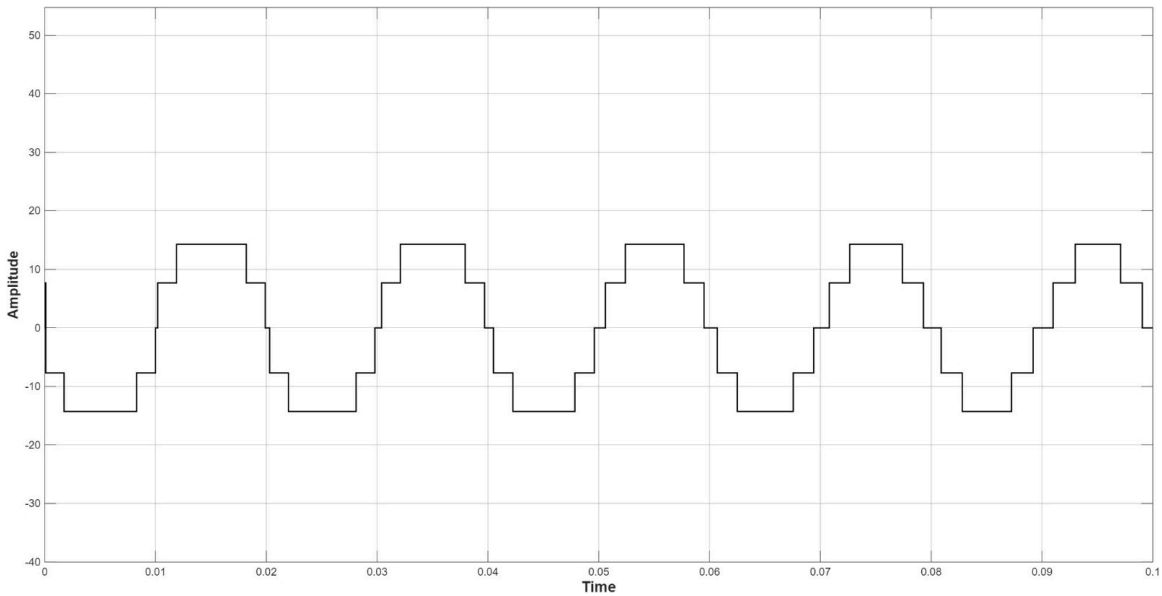
3. Add the following blocks: DC Voltage Source (2 or more), Universal Bridge (IGBT/Diode), PWM Generator, Series RLC Load, Voltage Measurement, Current Measurement, Scope, Powergui, and Ground.
4. For a cascaded H-bridge, use two H-bridge units with separate DC sources, connect their AC outputs in series, and obtain three output levels: +Vdc, 0, and –Vdc.
5. Use multi-carrier PWM by using 4 carrier signals for a 5-level inverter, comparing the sine reference with the triangular carriers via relational operators to generate gate pulses, and then connecting these pulses to the Universal Bridge.
6. Connect the converter output to an R or RLC load and add voltage and current measurement blocks.
7. Insert the Powergui block in discrete mode.
8. Save the file.
9. Click Run, open the Scope, and observe the output voltage waveform and output current waveform.

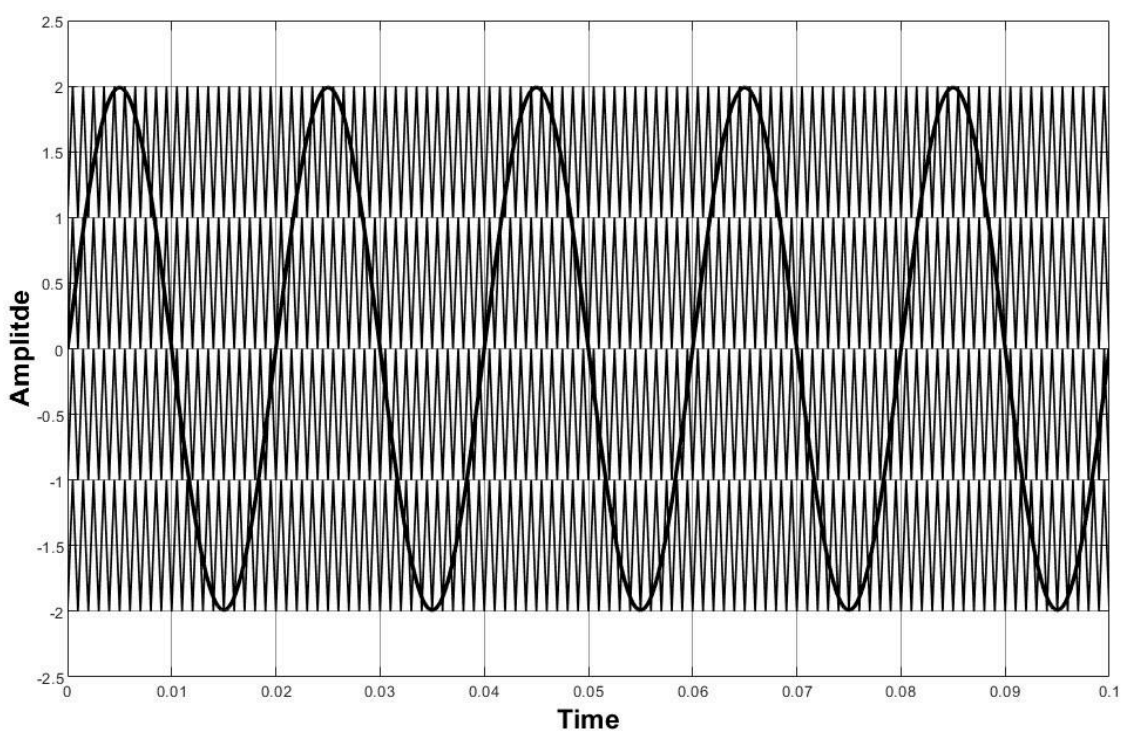
RESULTS:



CONCLUSION:

The multilevel inverter model produces stepped voltage waveforms that approximate a sinusoidal output. The simulation results show reduced harmonic distortion compared to conventional inverters. This confirms that multilevel inverters improve power quality and are suitable for high-power applications.





5. Solar PV Panel System

TITLE: Modelling of Solar PV Panel System using MATLAB Simulink.

INTRODUCTION:

A Solar Photovoltaic (PV) system converts solar energy into electrical energy using photovoltaic cells based on the photovoltaic effect. When sunlight falls on the semiconductor material of the solar cell, it generates direct current (DC) electricity. Multiple solar cells are connected to form a solar panel, and several panels together form a PV array to produce higher power.
In Simulink, the solar PV system is modelled using a PV array along with power electronic components such as converters, inverters, and loads to study system performance. The PV array generates DC power depending on solar irradiance and temperature, and this power can be converted into AC power using an inverter to supply electrical loads or connect to the grid. The simulation helps in analysing the voltage, current, and power characteristics of the solar PV system under different operating conditions.

A Solar Photovoltaic (PV) system with MPPT, Boost converter, battery storage, and a bidirectional DC–DC converter is used to efficiently convert solar energy into electrical energy and supply power to a DC load. The PV array converts solar radiation into DC electricity based on the photovoltaic effect. The output of the PV system varies with solar irradiance and temperature, so a Maximum Power Point Tracking (MPPT) technique is used to extract maximum power. In this system, a Fuzzy Logic MPPT controller processes the PV voltage and current to generate the required duty cycle, which is converted into switching pulses using a PWM generator. These pulses control the boost converter, which increases the PV output voltage to the required DC bus level. A battery energy storage system is connected through a bidirectional DC–DC converter, allowing power flow in both directions. When PV power is in excess, the battery charges, and when PV power is insufficient, the battery discharges to supply the load. The entire system is modelled in MATLAB/Simulink to analyse parameters such as voltage, current, power, and battery state of charge (SOC) under different operating conditions.

PROCEDURE:

Steps in the formation of the model:

The model mainly consists of PV Array block, Signal Editor block, Constant block, Bus Creator block, Voltage Measurement block, Current Measurement block, Product block, MATLAB Function block (Fuzzy MPPT controller), First-Order Filter block, PID Controller block, PWM Generator block, Data Type Conversion block, NOT Logic block, IGBT/Diode blocks, Diode block, Series RLC Branch blocks (inductor and capacitor), Battery block, Bidirectional DC-DC Converter blocks, Goto and From blocks, Display blocks, and Scope blocks for observing voltage, current, power, and battery parameters.

1. Open MATLAB. Type Simulink in the Command Window and click Blank Model to create a new Simulink model.

2. In the Library Browser, open Simscape → Electrical → Specialized Power Systems, Simulink → Sources, Simulink → Math Operations, Simulink → Sinks, and Simscape → Utilities.

3. Add the required blocks such as PV Array, Signal Editor, Constant, Bus Creator, Voltage Measurement, Current Measurement, Product, MATLAB Function block (Fuzzy MPPT controller), First-Order Filter, PID Controller, PWM Generator, Data Type Conversion, NOT Logic block, IGBT/Diode, Diode, Series RLC Branch (Inductor and Capacitor), Battery, Bidirectional DC-DC Converter, Goto/From blocks, Display, Scope, and Powergui.

4. Configure the PV Array by double-clicking the block and setting parameters such as irradiance, temperature, open-circuit voltage, short-circuit current, and number of modules, then click Apply.

5. Connect Voltage and Current Measurement blocks to the PV output and use a Product block to calculate the PV power.

6. Implement the MPPT controller by connecting the PV voltage and current signals to the MATLAB Function block (Fuzzy MPPT) to generate the required duty cycle.

7. Generate PWM pulses by feeding the duty cycle into the PWM Generator block, and apply these pulses to the IGBT switch of the Boost Converter.

8. Construct the Boost Converter using an inductor, diode, capacitor, and IGBT, and connect it to the PV array to increase the PV output voltage to the required DC bus level.

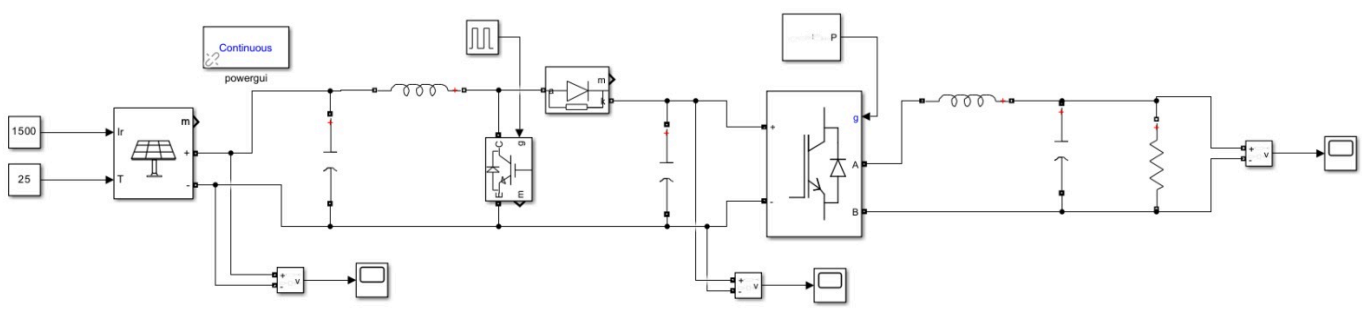
9. Connect the Battery through a Bidirectional DC-DC Converter so that excess PV energy can charge the battery and the battery can supply power when PV output is low.

10. Use a PID controller and filters to regulate the DC bus voltage and ensure stable system operation.

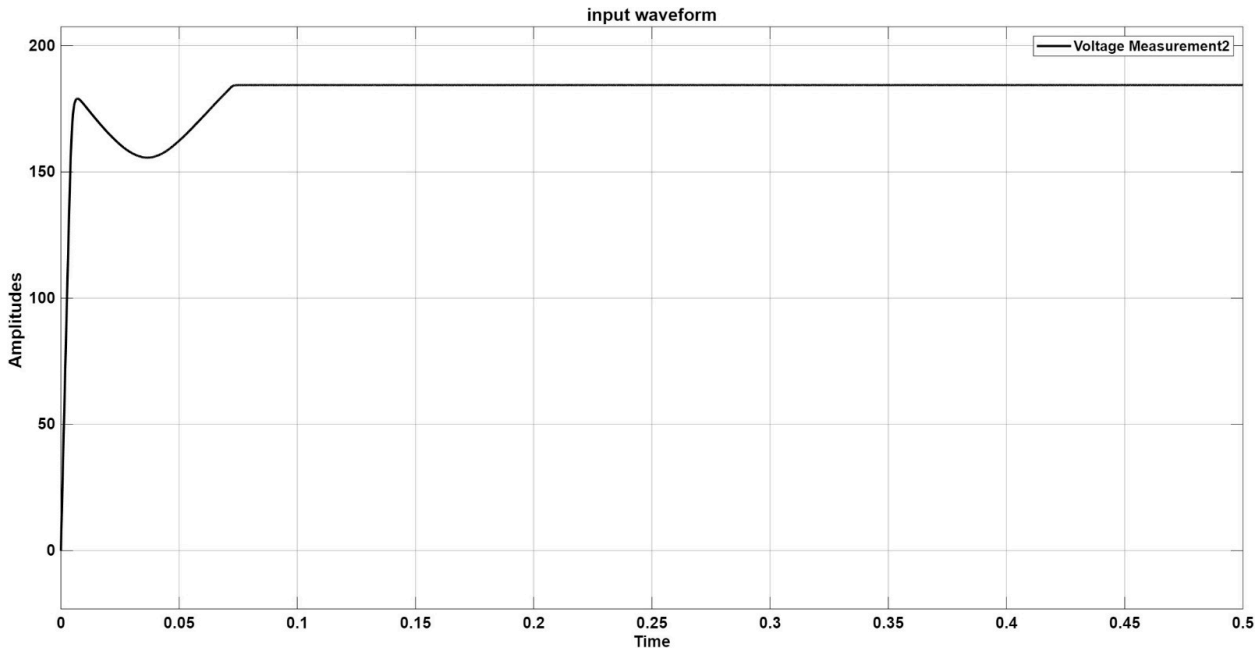
11. Connect the DC Load and attach the Display and Scope blocks to observe parameters such as PV voltage, PV current, output voltage, battery current, and power. Also, include the mandatory Powergui block in the model.

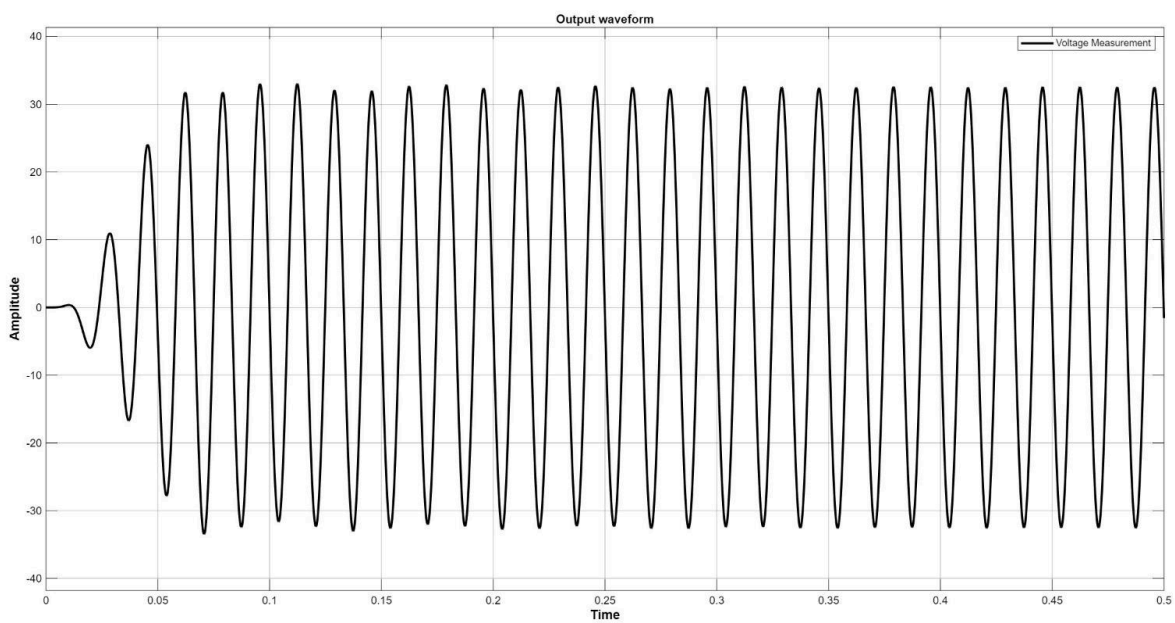
12. Save the model file, click Run, open the Scope, and observe the system responses such as voltage, current, and power characteristics of the solar PV system.

RESULTS:



CONCLUSION:





The simulation of the solar PV panel successfully illustrates the relationship between solar irradiance, temperature, and output voltage/current. The results show that the power output increases with increasing irradiance. This experiment verifies the electrical characteristics of a photovoltaic system and its behavior under different environmental conditions.

6. Open-loop and closed-loop position control of a DC motor

TITLE: Modelling of Open-loop and closed loop position control of DC motor using MATLAB Simulink.

INTRODUCTION:

Open-loop and closed-loop position control of a DC motor is used to control the angular position of the motor shaft. A DC motor converts electrical energy into mechanical rotation, and its position can be controlled by adjusting the input voltage. In an open-loop system, the input signal is applied directly to the motor without feedback. The motor moves according to the input command, but the actual position is not measured, so errors due to load or disturbances cannot be corrected. In a closed-loop system, a position sensor (encoder or potentiometer) measures the motor position and feeds it back to the controller. The controller compares the actual position with the reference position and reduces the error to achieve accurate position control. Closed-loop control provides higher accuracy and better stability compared to open-loop control.

PROCEDURE:

Steps in the formation of the model:

The model mainly consists of Constant (Input angle) block, Gain blocks (Position sensor gain, Attenuator gain, Potentiometer gain), Sum block, Transfer Function blocks (Electrical part and Mechanical part), Integrator block, Display blocks (Input voltage, Output voltage, Shaft motor position), and Scope blocks for observing the system response in both open-loop and closed-loop position control of the DC motor.

1. Open MATLAB. Type Simulink in the Command Window and click Blank Model to create a new Simulink model.

2. In the Library Browser, open Simulink → Sources, Simulink → Math Operations, Simulink → Continuous, Simulink → Sinks, and Simulink → Signal Routing.

3. Add the required blocks such as Constant (Input angle), Gain blocks, Sum block, Transfer Function blocks (Electrical part and Mechanical part), Integrator, Display, and Scope.

4. For the closed-loop system, connect the Constant block (input angle) to a Gain block (Position sensor gain) and then to a Sum block where the feedback signal will be subtracted from the reference input.

5. Connect the output of the Sum block to another Gain block (Attenuator gain) and then to the Transfer Function block representing the electrical part of the DC motor.

6. Connect the electrical part output to another Transfer Function block representing the mechanical part of the motor, and then connect it to an Integrator block to obtain the shaft position.

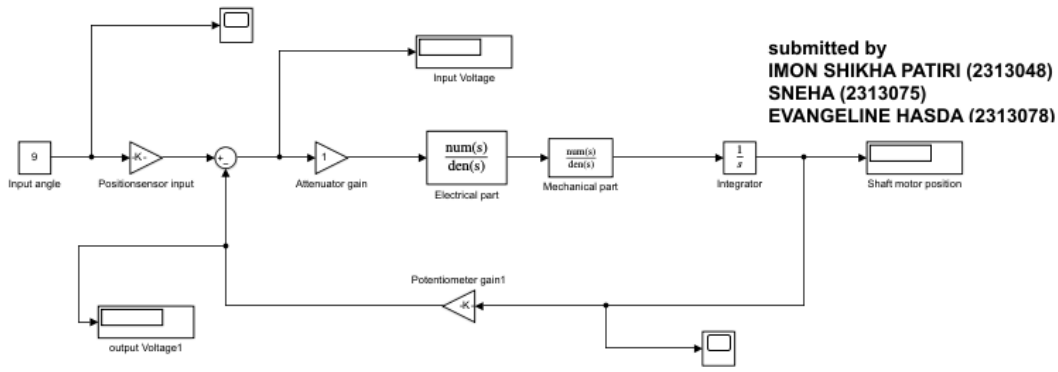
7. Connect the output (shaft motor position) through a Gain block (Potentiometer gain) back to the Sum block to form the feedback path, completing the closed-loop system.

8. For the open-loop system, connect the Constant block (input angle) directly through the Gain block (Attenuator gain) to the Transfer Function blocks (Electrical and Mechanical parts) and then to the Integrator, without any feedback connection.

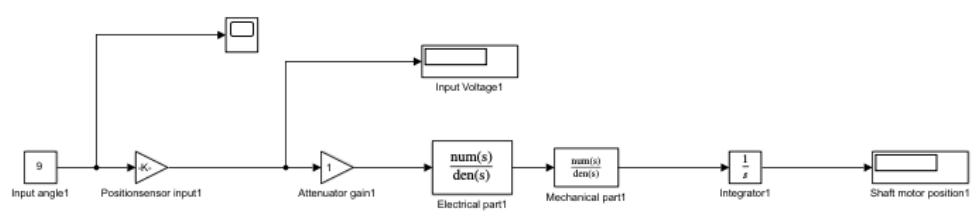
9. Connect the Display and Scope blocks to observe parameters such as input voltage, output voltage, and shaft motor position.

10. Save the model, click Run, and observe the system response on the Scope to compare the performance of open-loop and closed-loop position control of the DC motor.

RESULT:

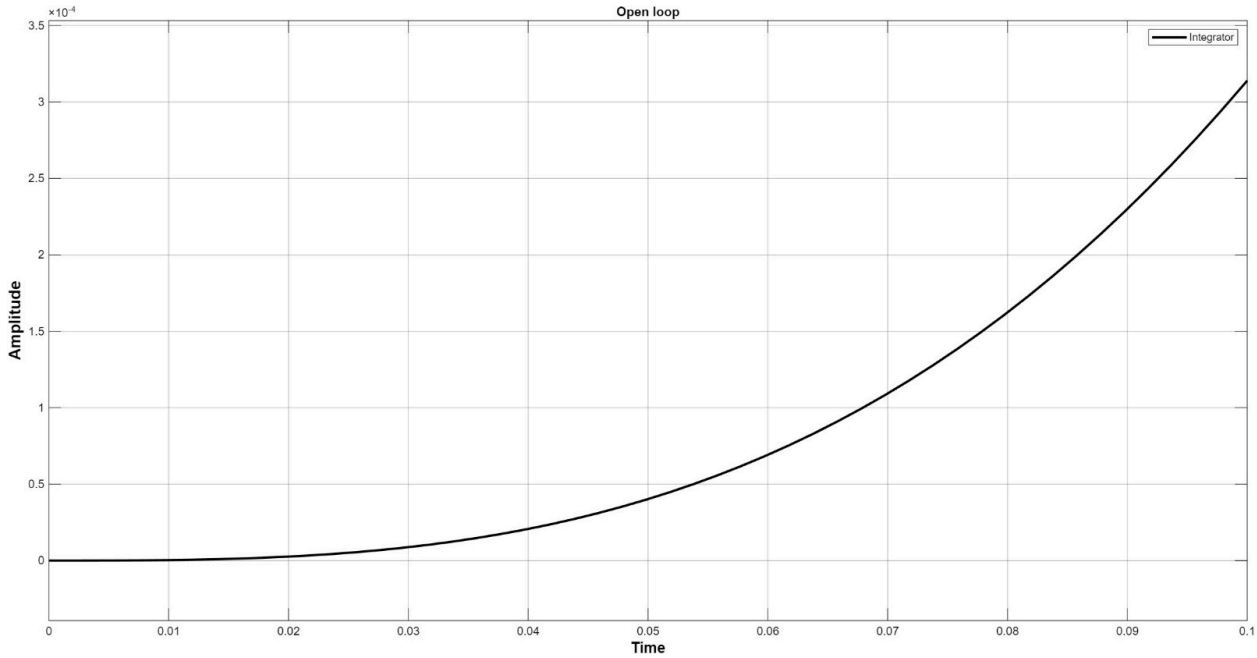


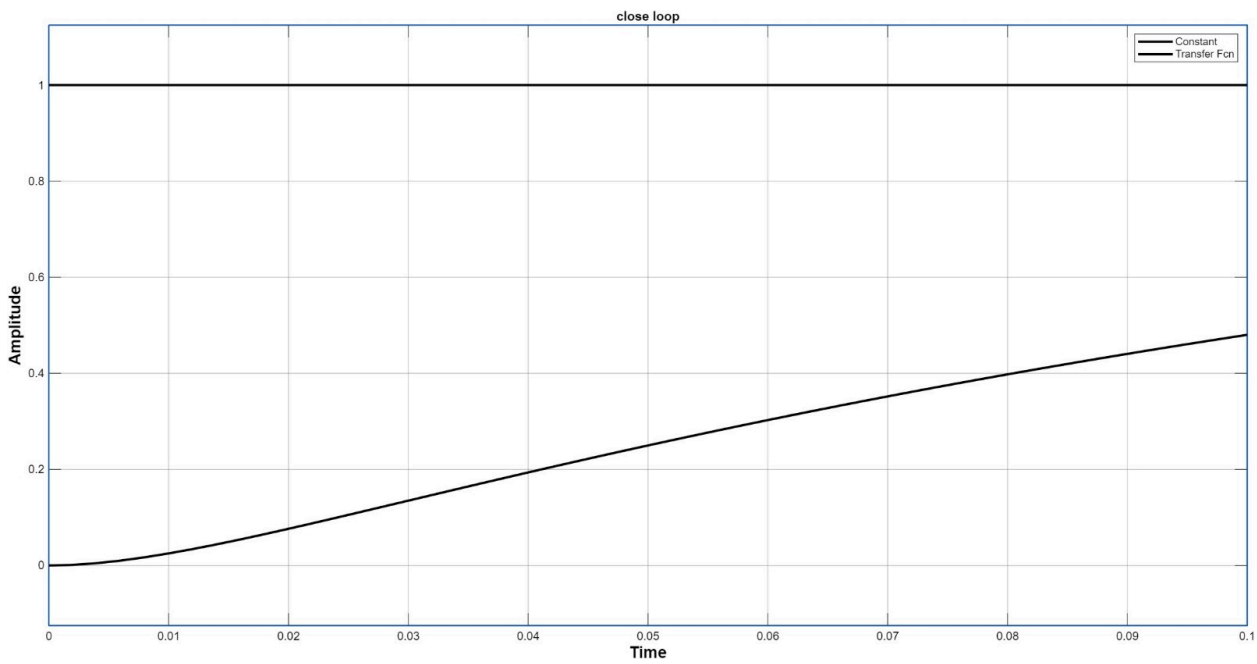
closed loop position control of dc motor



open loop position control
of dc motor

CONCLUSION:





The results show that the open-loop system has limited accuracy and is affected by disturbances. In contrast, the closed-loop system provides better stability, faster response, and improved accuracy due to feedback control. The simulation demonstrates the importance of feedback in achieving precise position control.

7. Wind Generator

TITLE: Modelling of Wind Generator using MATLAB Simulink.

INTRODUCTION:

A Wind Energy Conversion System (WECS) converts the kinetic energy of wind into electrical energy. When wind flows over the turbine blades, it causes the rotor to rotate, producing mechanical energy. This mechanical energy is transferred to a generator, which converts it into electrical power. In a wind energy system, the wind turbine captures energy from the wind and converts it into mechanical torque. This torque drives a generator, which produces electrical energy. The amount of power generated depends mainly on the wind speed, turbine blade design, and generator characteristics. The electrical power produced by the generator may be in AC form, which can be converted into DC or regulated voltage using power electronic converters for supplying loads or further processing. Filters are often used to reduce ripples and improve the quality of the output power. Wind energy systems are widely used because they are renewable, environmentally friendly, and reduce dependence on fossil fuels. Simulation of wind energy systems helps in analysing system performance under different wind speed conditions and load variations.

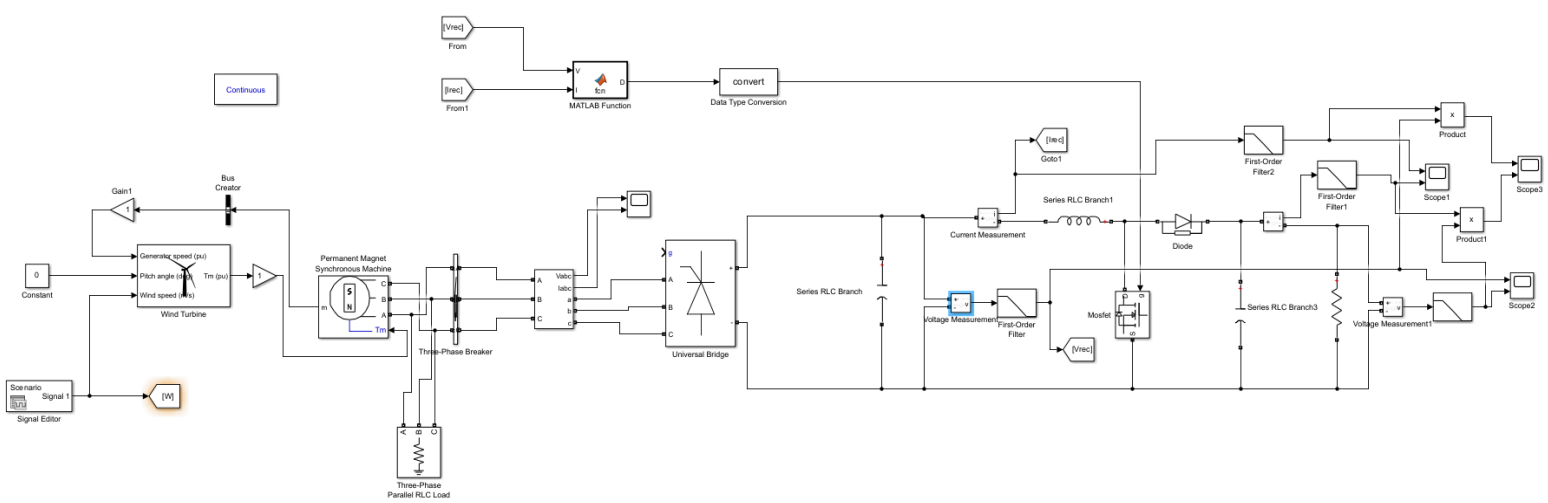
PROCEDURE:

Steps in the formation of the model: The model mainly consists of Powergui (Continuous), Constant block, Signal Editor block, Wind Turbine block, Permanent Magnet Synchronous Machine (PMSM), Bus Creator block, Three-Phase Breaker, Three-Phase Parallel RLC Load, Universal Bridge (rectifier), Series RLC Branch blocks, Voltage Measurement block, Current Measurement block, MOSFET switch, Diode, First-Order Filter blocks, MATLAB Function block, Data Type Conversion block, Product blocks, Goto/From blocks, and Scope blocks.

1. Open MATLAB.
2. Type Simulink in the Command Window and click Blank Model to create a new Simulink model.
3. Open the Library Browser and go to Simscape → Electrical → Specialized Power Systems, Simulink → Sources, Simulink → Math Operations, and Simulink → Sinks.

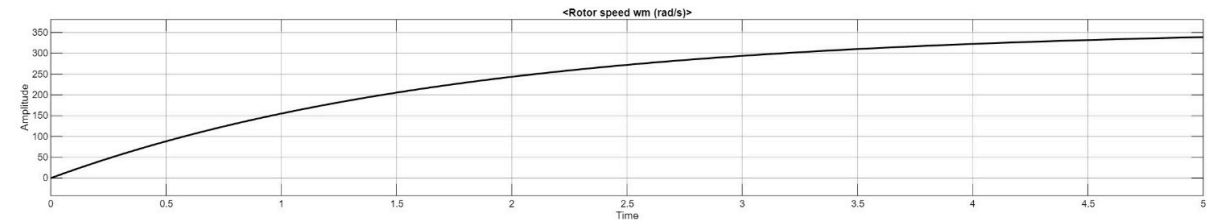
4. Add the required blocks such as Powergui, Constant, Signal Editor, Wind Turbine, Permanent Magnet Synchronous Machine (PMSM), Bus Creator, Three Phase Breaker, Universal Bridge, Series RLC Branch, Voltage Measurement, Current Measurement, MOSFET, Diode, First-Order Filter, MATLAB Function block, Product blocks, and Scope.
5. Configure the Wind Turbine block and connect its mechanical output to the PMSM generator.
6. Connect the three-phase output of the PMSM to the Three-Phase Breaker and then to the Universal Bridge rectifier to convert AC power into DC power.
7. Connect Series RLC Branch blocks at the rectifier output to act as a filter and reduce voltage ripple.
8. Implement a DC-DC converter using a MOSFET switch and a diode to regulate the DC output voltage.
9. Use Voltage Measurement and Current Measurement blocks along with Product blocks to measure voltage, current, and power.
10. Pass the measured signals through First Order Filter blocks and a MATLAB Function block to generate the switching control signal.
11. Connect Scope blocks to observe output voltage, current, and generated power. Save the model, click Run, and observe the waveforms on the Scope.

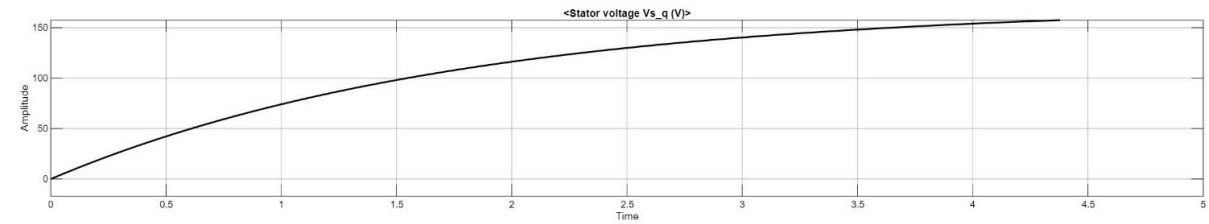
RESULT:

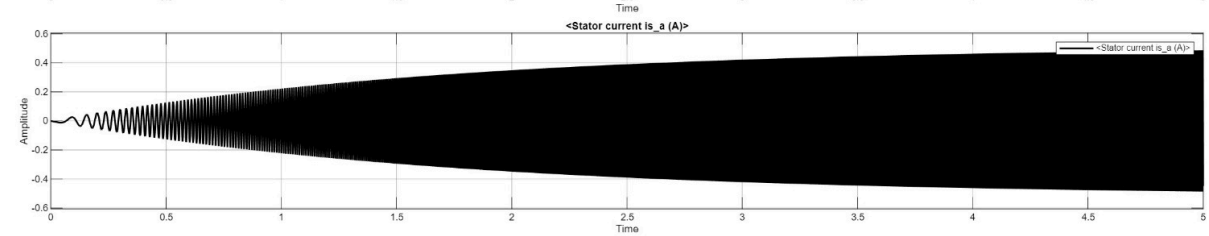


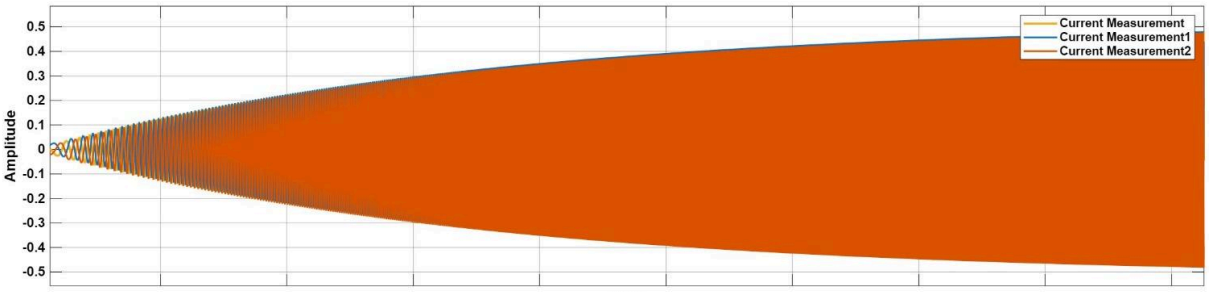
CONCLUSION:

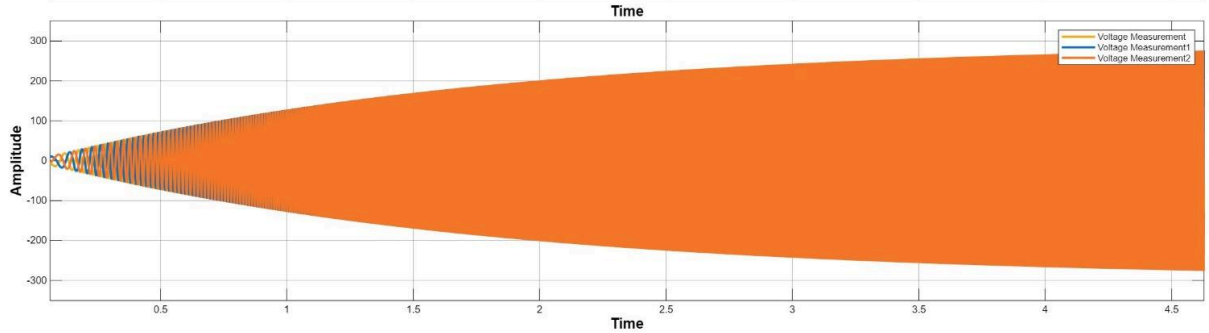
The wind generator model successfully simulates the conversion of wind energy into electrical power. The results show that generated power depends on wind speed, with higher wind speeds producing greater electrical output. This experiment highlights the effectiveness of wind energy systems for renewable power generation.











8. BUCK BOOST CONVERTER

TITLE: Modelling of buck and boost converter using MATLAB Simulink.

INTRODUCTION:

A Buck–Boost converter is a type of DC–DC converter used to obtain an output voltage that can be either higher or lower than the input voltage. It is widely used in power electronic applications such as battery-powered devices, DC motor drives, renewable energy systems, and regulated power supplies.The converter mainly consists of a DC voltage source, switch (MOSFET/IGBT), diode, inductor, capacitor, and load resistor. The switching action is controlled using a PWM pulse generator. When the switch is ON, energy is stored in the inductor. When the switch is OFF, the stored energy is transferred to the load through the diode and capacitor.By changing the duty cycle of the PWM signal, the output voltage can be controlled. The capacitor helps in smoothing the output voltage and reducing ripple. Voltage and current measurement blocks are used to observe the performance of the converter using scopes.

PROCEDURE:

Steps in the formation of the model

The model mainly consists of Powergui block, Pulse Generator block, Switch (IGBT/MOSFET) block, Diode block, Inductor block, Capacitor block, DC Voltage Source block, Load (Resistor) block, Voltage Measurement block, Current Measurement block, Mux block, and Scope.

1.Open MATLAB. Type Simulink in the Command Window and create a Blank Model.

2. Open the Simulink Library Browser and go to
Simscape → Electrical → Specialized Power Systems → Power Electronics and Elements.

3. Drag and place the Powergui block into the model and set it to continuous mode.

4. Add a DC Voltage Source block to provide the input voltage to the converter.

5. Insert a Pulse Generator block, which produces PWM pulses to control the switching device.

6. Place a Switch block (IGBT/MOSFET) and connect its gate terminal to the output of the pulse generator.

7. Add an Inductor block in series with the circuit to store energy during switching.

8. Insert a Diode block to provide the current path when the switch is OFF.

9. Connect a capacitor block across the output to smooth the output voltage.

10. Add a Load resistor block across the capacitor to represent the load.

11. Insert Voltage Measurement blocks at the input and output terminals to measure the voltages.

12. Add a Current Measurement block to measure the diode current.

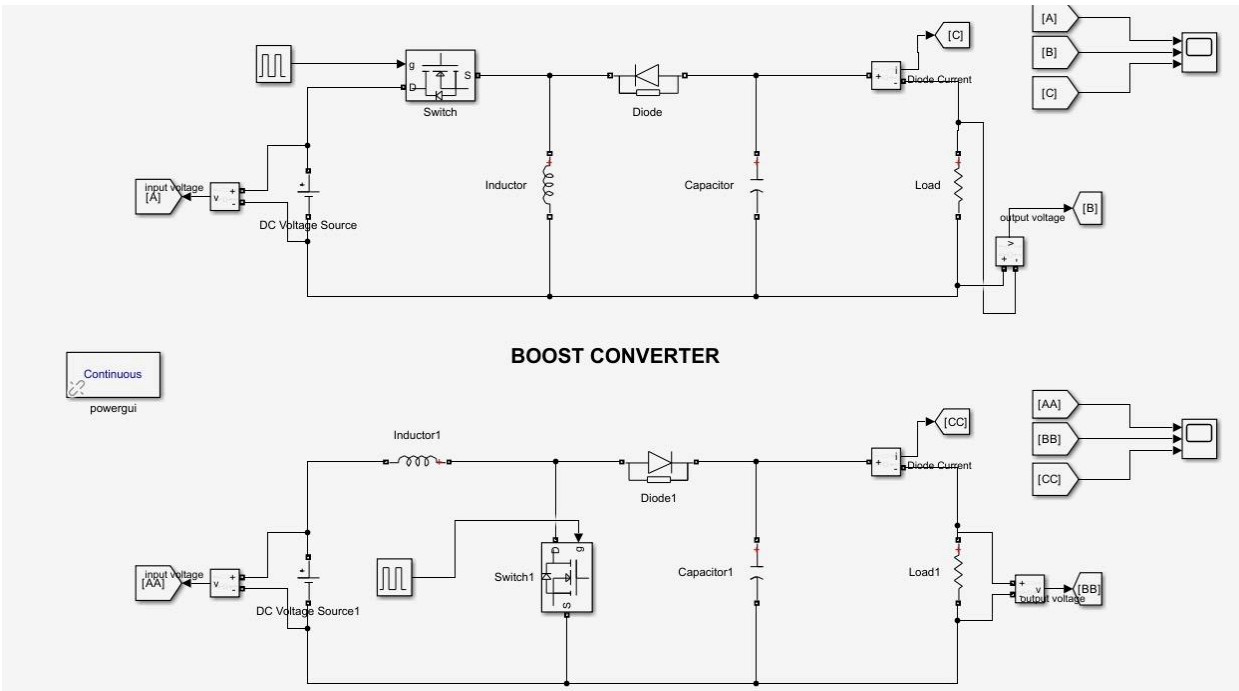
13.Use Mux blocks to combine multiple signals such as voltage and current.

14.Connect the Mux output to a Scope block to observe the input voltage, output voltage, and diode current.

15.Save the model.

16.Click Run and open the Scope to observe the waveforms of input voltage, output voltage, and diode current of the Buck Boost converter.

RESULT:



CONCLUSION:

The simulation of buck and boost converters verifies their ability to regulate output voltage. The buck converter reduces the input voltage while the boost converter increases it. The results confirm that these converters are essential components in power electronics for efficient voltage control.